

System Design: Instagram Feed

Social Media News Feed — Senior Engineer Interview Walkthrough

1. Requirements & Capacity

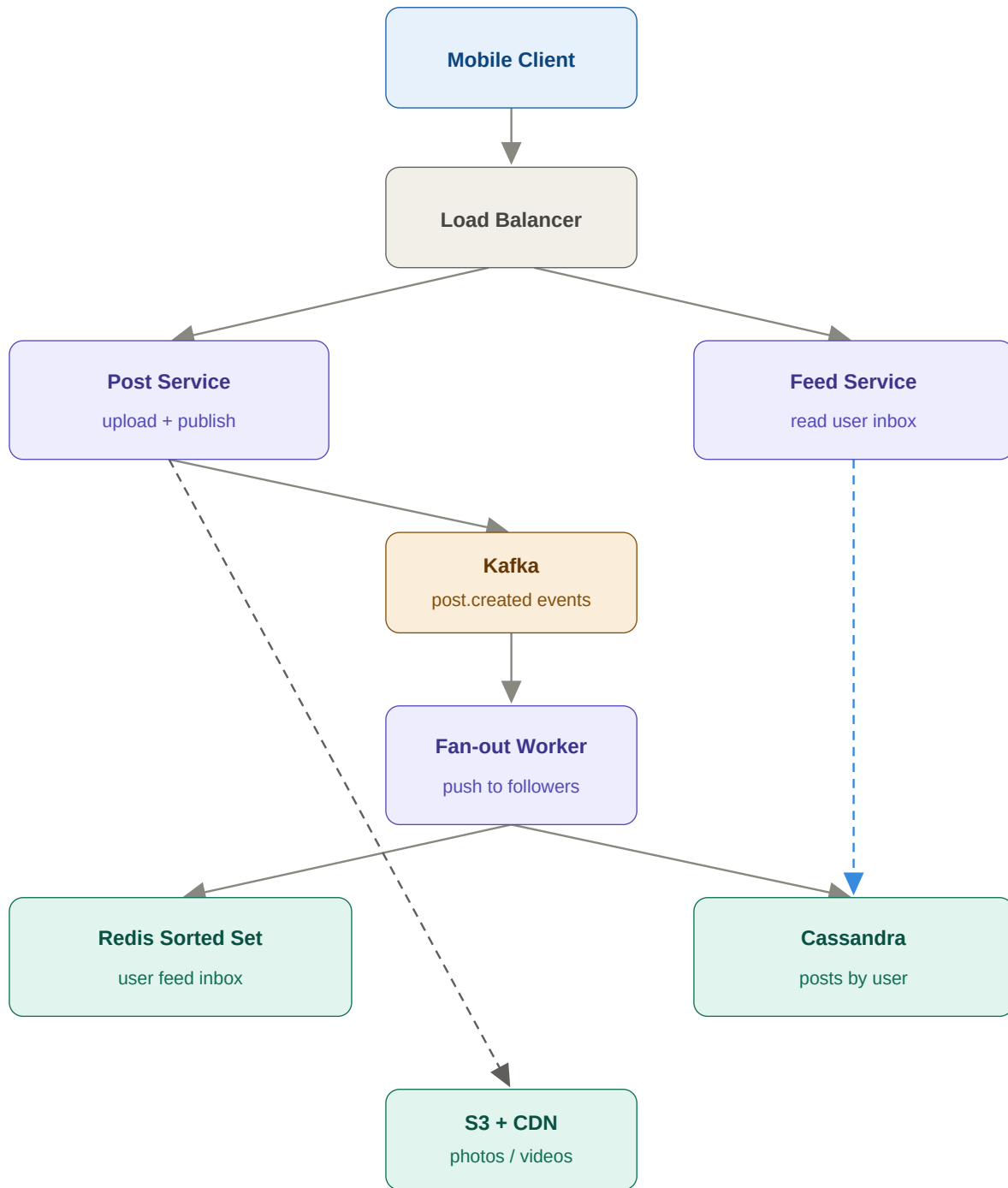
Think out loud: "Home feed (people I follow), infinite scroll, photos + videos, 500M DAU. How fresh must it be? 30s lag is fine."

- Functional: view feed, post photo/video, like, comment, follow/unfollow
- Non-functional: feed load < 200ms P99, eventual consistency OK (1s stale is fine)
- Scale: 500M DAU × 5 opens/day = 2.5B feed loads = ~29k RPS. 50M posts/day × 3MB = 150 TB/day media

Scale	Users/RPS	Architecture	Storage
Low	1M DAU 100 RPS	Monolith + Postgres + S3	~100 GB media/day
Mid	50M DAU 5k RPS	Microservices + Redis feed cache + S3 + CloudFront	~5 TB/day
High	500M DAU 29k RPS	Fan-out service + pre-computed feeds + Cassandra + CDN	~150 TB/day

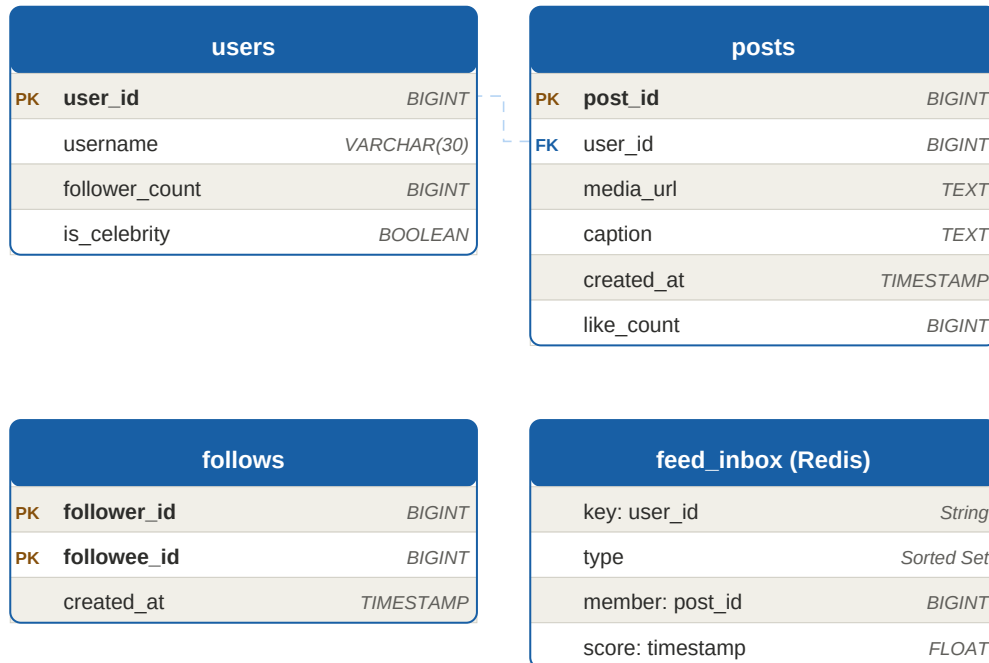
2. Architecture Diagram

Think out loud: "The core design decision: fan-out on write (push) vs fan-out on read (pull). Push pre-computes feeds. Pull computes at read time. At 500M users, push wins for reads, but celebrities need hybrid."



Architecture: write path (left) publishes post → Kafka → fan-out fills Redis inboxes. Read path (right) reads Redis in $O(1)$. Dashed = async.

3. DB Schema



Why this choice: Cassandra for posts: partition key = user_id, clustering = created_at DESC. Profile page query ("show me user X's posts") = single partition read, O(1). Handles petabyte scale.

Why Redis Sorted Set: Redis Sorted Set for feed inbox: ZADD user_feed score=timestamp member=post_id. ZREVRANGE gives latest posts instantly. Feed read = single Redis command.

4. Celebrity Problem & API

Think out loud: "Selena Gomez has 400M followers. Her one post = 400M Redis writes. That's our hot user problem. Solution: hybrid model."

- Celebrities (>1M followers): skip fan-out. Pull their posts at read time and merge.
- Regular users: full fan-out push on every post.
- Threshold stored on user record (is_celebrity=true). Fan-out service checks before processing.

GET	/api/v1/feed?cursor=&limit;=20	Cursor-paginated home feed
POST	/api/v1/posts	{media_presigned_key, caption} → {post_id}
POST	/api/v1/posts/{id}/like	Toggle like (async counter update)
POST	/api/v1/follows/{user_id}	Follow — triggers fan-out rebuild

Decision	Choice	Gain	Trade-off
Feed model	Push (fan-out write)	O(1) feed read	Write amplification
Posts DB	Cassandra	Petabyte scale	No ACID, no joins

Pagination	Cursor-based	Stable under inserts	Client stores cursor
Celebrity	Hybrid pull	No fan-out storm	Extra merge logic on read